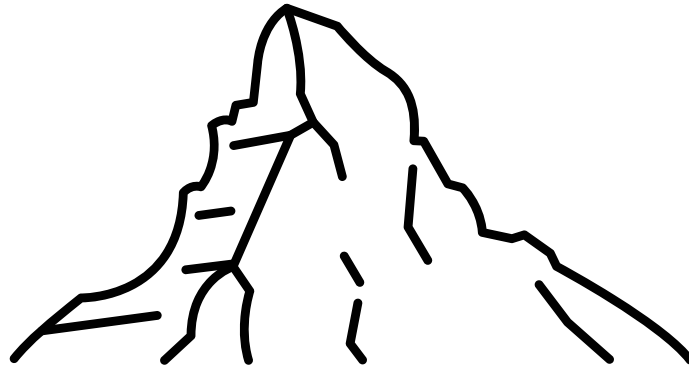


Piz Planer

Project Documentation Swiss Hiking Route Finder



Introduction to programming
(4,799,1.00)
University of St.Gallen

Group Project

PizPlaner

Project Documentation

Overview

The Pizplaner is a two-script Python project that scrapes hiking routes from SchweizMobil's public API and lets users interactively search for hikes that match their preferences, including starting location, drive time, duration, difficulty, and distance.

Project Structure

The project consists of two Python scripts that work in sequence:

scrape_schweizmobil.py:

- This is the program we used to fetch the Hiking routes of Switzerland. It calls the SchweizMobil API, filters routes, and writes the results to a local CSV file called routes.csv.

find_route.py:

- This the core of our project. The interactive program to find your hiking route. It reads routes.csv, asks the user for their preferences, geocodes trailhead locations via OpenStreetMap, and displays the best matching hikes.

Requirements

Python Version

Python 3.7 or higher is required.

Dependencies

The project uses one external library that is not included with Python by default: requests, which handles all HTTP calls to the SchweizMobil API and the OpenStreetMap Nominatim geocoding service. It must be installed before running either script. All other imports (csv, json, math, os, sys, time) are part of the Python standard library and require no installation.

Setup & First Run

1. **Install the dependency** (only needed once per environment). Install the requests library using pip before running the scripts for the first time.
2. **Fetch the route data** by running `scrape_schweizmobil.py`. This creates `routes.csv` in the same folder. The script prints progress as it runs and typically collects several hundred routes. Runtime is a few minutes due to polite delays between API calls.

Alternatively, a pre-fetched `routes.csv` and `geocode_cache.json` are included in the project folder. If you want to skip the data fetching step entirely, you can proceed directly to Step 3 using these files.

3. **Search for hikes** by running `find_route.py`. The script asks a series of questions (location, drive time, trip length, etc.) and then displays matching hikes with links to SchweizMobil.

Script 1: `scrape_schweizmobil.py`

What it does

Queries the SchweizMobil JSON API across national, regional, and local categories and collects hikes that meet the following criteria: 1 to 3 days long (longer multi-stage routes are skipped), a total distance of at least 8 km, and no single day longer than 10 hours.

For each qualifying route it saves: name, region (start → end), distance, elevation gain, difficulty, estimated duration, number of days, a short description, route number, and a direct link to the SchweizMobil website.

Output

A file called `routes.csv` in the working directory, with up to 500 routes.

Key Configuration

The following settings can be adjusted at the top of the script:

Variable	Default	Meaning
<code>MIN_DISTANCE_KM</code>	8	Minimum total route distance in km
<code>MAX_DAY_DURATION_MIN</code>	600	Maximum duration of one day in minutes
<code>MAX_DAYS</code>	3	Maximum number of days per route
<code>MAX_RESULTS</code>	500	Hard cap on collected routes
<code>LANG</code>	"de"	API response language (de, en, fr, it)

Duration Estimation

The script uses the `hikingTime` field from the API when available. If that field is missing, it falls back to Naismith's Rule: 1 hour per 5 km of distance, plus 1 hour per 600 m of elevation gain.

Script 2: find_route.py

What it does

Loads routes.csv and guides the user through a series of filter questions. It then geocodes the user's starting location and each trailhead via OpenStreetMap, estimates drive time using straight-line distance divided by an average speed of 50 km/h, and returns the best-matching hikes sorted by how closely the daily duration matches the user's target.

Geocoding Cache

To avoid repeating API calls, all geocoded coordinates are saved to geocode_cache.json. On subsequent runs, cached results are used instantly without hitting the network. This file is created automatically and can be safely deleted to force a fresh lookup.

Interactive Prompts

Prompt	What it filters
Starting location	City or address used to estimate drive time
Max drive time	Trailheads reachable within this many minutes
Trip length	Day hike (1 d), weekend (2 d), long weekend (3 d), or any
Target daily duration $\pm$ tolerance	Hikes where each day falls within this range
Difficulty	Wanderweg, Bergwanderweg, Alpinwanderweg, or any
Max distance	Optional cap on total route distance
Number of results	How many matches to display per search

After seeing results, the user can show more matches, tweak the search criteria, or quit.

Difficulty Levels (Swiss SAC Trail System)

Type	Description
Wanderweg	Well-marked yellow trails, suitable for everyone
Bergwanderweg	Mountain trails, requires sure-footedness
Alpinwanderweg	Alpine terrain, experience and good equipment required

Data Flow

The two scripts connect as follows: the scraper fetches data from the SchweizMobil API and writes it to routes.csv. The finder reads that file, contacts the OpenStreetMap Nominatim service to geocode locations, persists those results in geocode_cache.json, and outputs hike recommendations to the terminal.

Limitations & Notes

- Drive time is approximate. The estimate uses straight-line distance divided by 50 km/h and does not account for actual road routing, mountain passes, or traffic.
- Geocoding accuracy depends on place names. Trailhead names from SchweizMobil are not always recognized by Nominatim; routes where the start location cannot be geocoded are silently skipped in the finder.
- API availability. The scraper relies on SchweizMobil's public API. If the API changes its structure or becomes unavailable, the scraper may need to be updated.
- Re-run the scraper periodically. New routes are added to SchweizMobil over time; deleting routes.csv and re-running the scraper refreshes the dataset.
- If filter is set wide, geocoding the locations takes time.
- The biggest part of the data set will be day hikes and not multiday hikes, so the PizPlaner will be most useful for these type of hikes.